Practical : 9	Implementation of Prim's Algorithm
Date : __ / __ / 2026	

Aim: To implement **Prim's algorithm** in C++ to find the **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph.

Objective :- To store a weighted graph as an adjacency list, build the MST using a min-heap (priority queue), and to analyze the time and space complexity of the algorithm.

A. Graph Representation (Weighted Adjacency List)

1. Start.
2. Read the number of vertices V and the number of edges E.
3. Create an array of V lists; each list stores pairs (neighbour, weight).
4. For each edge (u, v, w), add (v, w) to the list of u and (u, w) to the list of v.
5. Display the adjacency list.
6. Stop.

B. Prim's Algorithm

1. Start.
2. Read the starting vertex s.
3. Create key[] (all infinity), parent[] (all -1), inMST[] (all false) and an empty min-heap.
4. Set key[s] = 0 and push (0, s) into the min-heap.
5. While the heap is not empty, remove the pair with the smallest key; let its vertex be u.
6. If u is already in the MST, skip it. Otherwise mark inMST[u] as true.
7. For every neighbour v of u with edge weight w: if v is not in the MST and w < key[v], set key[v] = w, parent[v] = u and push (key[v], v) into the heap.
8. Repeat steps 5 to 7 until the heap becomes empty.
9. Print every edge (parent[v], v) with its weight, and the total weight of the MST.
10. Stop.

Program A: Prim's Algorithm (practical9_prim.cpp)

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;
class Graph {
    int V;
    vector<vector<pair<int, int>>> adj;   // adj[u] = list of (v, weight)
public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }
    // Add an undirected weighted edge
    void addEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    // Display adjacency list
    void displayGraph() {
        cout << "\n--- Adjacency List (vertex, weight) ---" << endl;
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (auto &p : adj[i]) {
                cout << "(" << p.first << ", " << p.second << ") ";
            }
            cout << endl;
        }
    }
    // Prim's Algorithm (uses Min-Heap / Priority Queue)
    void primMST(int start) {
        vector<int> key(V, INT_MAX);      // cheapest edge weight to reach each vertex
        vector<int> parent(V, -1);        // parent of each vertex in the MST
        vector<bool> inMST(V, false);
        // min-heap of (key, vertex)
        priority_queue<pair<int, int>, vector<pair<int, int>>,
                       greater<pair<int, int>>> pq;
        key[start] = 0;
        pq.push({0, start});
        while (!pq.empty()) {
            int u = pq.top().second;
            pq.pop();
            if (inMST[u]) continue;       // skip outdated heap entries
            inMST[u] = true;
            for (auto &p : adj[u]) {
                int v = p.first;
                int w = p.second;
                if (!inMST[v] && w < key[v]) {
                    key[v] = w;
                    parent[v] = u;
                    pq.push({key[v], v});
                }
            }
        }
        cout << "\n--- Minimum Spanning Tree (Prim's) ---" << endl;
        cout << "Edge\tWeight" << endl;
        int total = 0, count = 0;
        for (int v = 0; v < V; v++) {
            if (parent[v] != -1) {
                cout << parent[v] << " - " << v << "\t" << key[v] << endl;
                total += key[v];
                count++;
            }
        }
        if (count < V - 1) {
            cout << "Graph is not connected, so a full spanning tree is not possible." << endl;
        }
        cout << "Total weight of MST = " << total << endl;
```

```
    }
};
int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;
    Graph g(V);
    cout << "Enter edges (u v weight):" << endl;
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u < 0 || u >= V || v < 0 || v >= V) {
            cout << "Invalid edge. Vertices must be between 0 and "
                 << V - 1 << "." << endl;
            i--;
            continue;
        }
        g.addEdge(u, v, w);
    }
    int start;
    cout << "Enter starting vertex: ";
    cin >> start;
    if (start < 0 || start >= V) {
        cout << "Invalid starting vertex." << endl;
        return 0;
    }
    g.displayGraph();
    g.primMST(start);
    return 0;
}
```

Output:

Sample graph used: 6 vertices (0 to 5) and 9 edges (u-v: weight): 0-1: 4, 0-2: 3, 1-2: 1, 1-3: 2, 2-3: 4, 3-4: 2, 4-5: 6, 3-5: 7, 2-4: 5. Starting vertex: 0. The MST has V - 1 = 5 edges and total weight 14.

```
Output

Enter number of vertices: 6
Enter number of edges: 9
Enter edges (u v weight):
0 1 4
0 2 3
1 2 1
1 3 2
2 3 4
3 4 2
4 5 6
3 5 7
2 4 5
Enter starting vertex: 0

--- Adjacency List (vertex, weight) ---
0 -> (1, 4) (2, 3)
1 -> (0, 4) (2, 1) (3, 2)
2 -> (0, 3) (1, 1) (3, 4) (4, 5)
3 -> (1, 2) (2, 4) (4, 2) (5, 7)
4 -> (3, 2) (5, 6) (2, 5)
5 -> (4, 6) (3, 7)

--- Minimum Spanning Tree (Prim's) ---
Edge    Weight
2 - 1   1
0 - 2   3
1 - 3   2
3 - 4   2
4 - 5   6
Total weight of MST = 14


=== Code Execution Successful ===
```

Program B: Performance Analysis (practical9_analysis.cpp)

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
#include <algorithm>
#include <random>
#include <chrono>
using namespace std;
using namespace chrono;
class Graph {
    int V;
    vector<vector<pair<int, int>>> adj;
public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }
    void addEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
```

```
        adj[v].push_back({u, w});
    }
    // Prim's Algorithm (Min-Heap); returns total weight of the MST
    long long primMST(int start) {
        vector<int> key(V, INT_MAX);
        vector<bool> inMST(V, false);
        priority_queue<pair<int, int>, vector<pair<int, int>>,
                       greater<pair<int, int>>> pq;
        key[start] = 0;
        pq.push({0, start});
        long long total = 0;
        while (!pq.empty()) {
            int u = pq.top().second;
            pq.pop();
            if (inMST[u]) continue;
            inMST[u] = true;
            total += key[u];
            for (auto &p : adj[u]) {
                int v = p.first, w = p.second;
                if (!inMST[v] && w < key[v]) {
                    key[v] = w;
                    pq.push({key[v], v});
                }
            }
        }
        return total;
    }
};
// Build a connected random graph with V vertices and E edges (fixed seed)
Graph generateGraph(int V, int E) {
    mt19937 rng(42);
    Graph g(V);
    for (int i = 1; i < V; i++) {                 // spanning chain keeps it connected
        int j = rng() % i;
        g.addEdge(i, j, rng() % 100 + 1);
    }
    for (int i = V - 1; i < E; i++) {             // remaining random edges
        int u = rng() % V, v = rng() % V;
        if (u == v) { i--; continue; }
        g.addEdge(u, v, rng() % 100 + 1);
    }
    return g;
}
int main() {
    int sizes[] = {100, 500, 1000, 5000, 10000, 50000, 100000};
    cout << "Prim's Algorithm Performance Analysis" << endl << endl;
    for (int V : sizes) {
        int E = 5 * V;
        Graph g = generateGraph(V, E);
        long long weight = 0, us = LLONG_MAX;
        for (int run = 0; run < 3; run++) {          // best of 3 runs
            auto start = high_resolution_clock::now();
            weight = g.primMST(0);
            auto stop = high_resolution_clock::now();
            us = min(us, (long long)duration_cast<microseconds>(stop - start).count());
        }
        cout << "Number of Vertices = " << V << endl;
        cout << "Number of Edges    = " << E << endl;
        cout << "MST Total Weight   : " << weight << endl;
        cout << "Prim's Time        : " << us << " microseconds" << endl;
        cout << "----------------------------------------" << endl;
    }
    return 0;
}
```

Output:

Random connected graphs with V vertices and E = 5V edges (weights 1 to 100) were used for V = 100, 500, 1000, 5000, 10000, 50000 and 100000. Each time is the best of 3 runs.

```
Prim's Algorithm Performance Analysis

Number of Vertices = 100
Number of Edges    = 500
MST Total Weight   : 1217
Prim's Time        : 155 microseconds
---------------------------------------
Number of Vertices = 500
Number of Edges    = 2500
MST Total Weight   : 5983
Prim's Time        : 876 microseconds
---------------------------------------
Number of Vertices = 1000
Number of Edges    = 5000
MST Total Weight   : 12478
Prim's Time        : 1956 microseconds
---------------------------------------
Number of Vertices = 5000
Number of Edges    = 25000
MST Total Weight   : 62064
Prim's Time        : 11217 microseconds
---------------------------------------
Number of Vertices = 10000
Number of Edges    = 50000
MST Total Weight   : 125388
Prim's Time        : 23075 microseconds
---------------------------------------
Number of Vertices = 50000
Number of Edges    = 250000
MST Total Weight   : 624058
Prim's Time        : 149637 microseconds
---------------------------------------
Number of Vertices = 100000
Number of Edges    = 500000
MST Total Weight   : 1249614
Prim's Time        : 308077 microseconds
---------------------------------------
```

Observation:

When V and E grew 1000 times (V = 100 to V = 100000), the time of Prim's algorithm grew from 155 to 308077 microseconds, about 1988 times. For the same change, E log V grows about 2500 times, so the measured growth agrees with the theoretical O(E log V) and is far below quadratic growth.

The exact timings depend on the computer and compiler, so a different machine will show different numbers, but the growth pattern stays the same.

Analysis:

Time Complexity Analysis

Let V be the number of vertices and E the number of edges of the graph.

Prim's Algorithm (adjacency list + min-heap):

Every vertex is added to the MST exactly once. Every edge is examined at most twice (once from each end), so the heap receives at most O(E) entries.

Each push or pop on the heap costs O(log E), which is O(log V) because E is at most V^2.

Time Complexity = O(E log V)

Note: if an adjacency matrix is used with a simple array scan instead of a heap, the time becomes $O(V^2)$, which is better for very dense graphs. With a Fibonacci heap it improves to O(E + V log V).

Space Complexity Analysis

The arrays key[], parent[] and inMST[] use O(V) space.

The heap may hold outdated entries (lazy deletion), so it can contain up to O(E) pairs.

Space Complexity = O(V + E)

The adjacency list itself also needs O(V + E) space to store the graph.

Prim's Algorithm with Different Data Structures

Feature	Adjacency matrix + array	Adjacency list + binary heap (used here)	Adjacency list + Fibonacci heap
Time complexity	$O(V^2)$	O(E log V)	O(E + V log V)
Best suited for	Dense graphs	Sparse graphs	Very large dense graphs
Implementation	Very simple	Simple (priority_queue)	Complex
Space	$O(V^2)$	O(V + E)	O(V + E)

Conclusion :-

Prim's algorithm builds the Minimum Spanning Tree by growing a single tree from the starting vertex. At every step it greedily adds the cheapest edge that connects a vertex inside the tree to a vertex outside it.

1. The program stores the graph as an adjacency list and uses a **min-heap**, giving **O(E log V)** time, which is efficient for sparse graphs.

2. For the sample graph the MST contains V - 1 = 5 edges with a minimum total weight of **14**.

3. The algorithm only spans the component of the starting vertex, so on a disconnected graph the program reports that a full spanning tree is not possible.